Project 2 HTTP based SQL Server (CS406:Spr 2025)
A Two-Person Group Project
Due: April 27th, 2026

1 Overview

In this assignment, you will be developing a SQL server based through a simple web server. To simplify this project, you are provided with the code for a very simple functional non-concurrent web server. In this project you will use a CGI mechanism to server a SQL database that is stored in a single structured file and not active memory.

This project will also be the basis for the next step, where you will alter the server to a concurrent system.

A good way to approach solving the problem is to first understand the code. Also, you can run this from your machine locally. There is often a challenge with ports.

The goals of this project are:

1. To learn the basic architecture of a simple web server
2. Build a simple SQL server using the Common Gateway Interface (CGI) mechanism.
3. Use a single file to store a table information, with a main schema table and files for the data of each table. Each file will consist of 245 bytes blocks that are chained and indexed using the `lseek` command.

The project will be performed in pairs and project components will be documented in the final report. The components will be divided between the team members, and this will be documented in the report. The project architecture, i.e. component design, will be a large percentage of the project grade.

Unit-testing for the created CGI must be implemented as a compiled part of the executable, while testing through the webserver will be a separate set of tests run as shell scripts. These tests must verify the required functionality of the SQL server. Also, the final report needs to provide a detailed test plan and show how the results to verify the functioning software by running the unit-tests. There should be a detailed description how your Professor can execute your tests and test plan.

2 HTTP Background and Provided Server

Before describing what you will be implementing in this project, a very brief overview of how a classic web server works is provided, and the HTTP protocol (version 1.0) used to communicate with it; although web browsers and servers have evolved over the years the old versions still work and give you a good start in understanding how things work. Our goal in providing you with a basic web server is that you can be shielded from learning all of the details of network connections and the HTTP protocol needed to do the project; however, the network code has been greatly simplified and is fairly understandable should you choose to to study it.

link to history lesson (click here)

`https://developer.mozilla.org/en-US/docs/Web/HTTP/Basics_of_HTTP/Evolution_of_HTTP`

Classic web browsers and web servers interact using a text-based protocol called `HTTP` (`Hypertext Transfer Protocol`). A web browser opens a connection to a web server and requests some content with HTTP. The web server responds with the requested content and closes the connection. The browser reads the content and displays it on the screen.

HTTP is built on top of the `TCP/IP` protocol suite provided by the operating system. Together, TPC and IP ensure that messages are routed to their correct destination, get from source to destination reliably in the face of failure, and do not overly congest the network by sending too many messages at once, among other features. To learn more about networks you can look to this book https://book.systemsapproach.org.

Each piece of content on the web server is associated with a file in the server's file system. The simplest is *static* content, in which a client sends a request just to read a specific file from the server. Slightly more complex is *dynamic* content, in which a client requests that an executable file be run on the web server and its output returned to the client. Each file has a unique name known as a `URL` (`Universal Resource Locator`).

As a simple example, let's say the client browser wants to fetch static content (i.e., just some file) from a web server running on some machine. The client might then type in the following URL to the browser: `http://www.justanexample.org/index.html`. This URL identifies that the HTTP protocol is to be used, and that an HTML file in the root directory (`/`) of the web server called `index.html` on the host machine `www.justanexample.org` should be fetched.

The web server is not just uniquely identified by which machine it is running on but also the `port` it is listening for connections upon. Ports are a communication abstraction that allow multiple (possibly independent) network communications to happen concurrently upon a machine; for example, the web server might be receiving an HTTP request upon port 80 while a mail server is sending email out using port 25. By default, web servers are expected to run on port 80 (the well-known HTTP port number), but sometimes (as in this project), a different port number will be used. To fetch a file from a web server running at a different port number (say 8000), specify the port number directly in the URL, e.g., `http://www.justanexample.org:8000/index.html`.

URLs for executable files (i.e., dynamic content) can include program arguments after the file name. For example, to just run a program (`test.cgi`) without any arguments, the client might use the URL `http://www.justanexample.org/test.cgi`. To specify more arguments, the `?` and `&` characters are used, with the `?` character to separate the file name from the arguments and the `&` character to separate each argument from the others. For example, `http://www.justanexample.org/test.cgi?x=10&y=20}` can be used to send multiple arguments `x` and `y` and their respective values to the program `test.cgi`. The program being run is called a **CGI program** (short for Common Gateway Interface). Details can be found here: Wikipedia Link.

`https://en.wikipedia.org/wiki/Common_Gateway_Interface`

The arguments are passed into the program as part of the **QUERY_STRING** environment variable, which the program can then parse to access these arguments. Details can be found here: Wikipedia Link.

`https://en.wikipedia.org/wiki/Query_string`

2.1 The HTTP Request

When a client (e.g., a browser) wants to fetch a file from a machine, the process starts by sending a machine a message. But what exactly is in the body of that message? These *request contents*, and the subsequent *reply contents*, are specified precisely by the HTTP protocol.

Let's start with the request contents, sent from the web browser to the server. This HTTP request consists of a request line, followed by zero or more request headers, and finally an empty text line. A request line has the form: **method uri version**. The **method** is usually `GET`, which tells the web server that the client simply wants to read the specified file; however, other methods exist (e.g., `POST`). The **uri** is the file name, and perhaps optional arguments (in the case of dynamic content). Finally, the 'version' indicates the version of the HTTP protocol that the web client is using (e.g., HTTP/1.0).

The HTTP response (from the server to the browser) is similar; it consists of a response line, zero or more response headers, an empty text line, and finally the interesting part, the response body. A response line has the form version **status message**. The **status** is a three-digit positive integer that indicates the state of the request; some common states are **200** for `OK`, **403** for `Forbidden` (i.e., the client can't access that file), and **404** for **File Not Found** (the famous error). Two important lines in the header are **Content-Type**, which tells the client the type of the content in the response body (e.g., HTML or gif or otherwise) and **Content-Length**, which indicates the file size in bytes.

For this project, you don't really need to know this information about HTTP unless you want to understand the details of the code we have given you. You will not need to modify any of the procedures in the web server that deal with the HTTP protocol or network connections. However, it's always good to learn more, isn't it?

2.2 A Basic Web Server

The code for the web server is available in this repository. You can compile the files herein by simply typing `make`. Compile and run this basic web server before making any changes to it. `make clean` removes `.o` files and executables, then performs a clean build.

When you run this basic web server, you need to specify the port number that it will listen on; ports below number 1024 are *reserved* (see the list at the link below) or this link.

`https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml`

For your project so you should specify port numbers that are greater than 1023 to avoid this reserved range; the max is 65535. Be wary: if running on a shared machine, you could conflict with others and thus have your server fail to bind to the desired port. If this happens, try a different number! But you will most likely be running on your local machine.

If you do run into issues with the port number, this might be an ITS issue. Please let the professor know if you run into the problem of things just not working.

When you then connect your web browser to this server, make sure that you specify this same port. For example, assume that you are running on the server `138.147.9.135` and use port number 8003; copy your favorite HTML file to the directory that you start the web server from. Then, to view this file from a web browser (running on the same or a different machine), use the url `138.147.9.135:8003/favorite.html`. If you run the client and web server on the same machine, you can just use the hostname `localhost` as a convenience, e.g., `localhost:8003/favorite.html`.

To find the current ip on the server, you can do this with the `ifconfig` command. On the server, you will see the following result. The first entry '`ens18`' shows the ethernet entry, which is `139.147.9.135`. The next entry is for local host, which happens to be the ip `127.0.0.1`

```
$ ifconfig
ens18: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 139.147.9.135  netmask 255.255.255.128  broadcast 139.147.9.255
        ether 2a:a3:0e:b3:bd:2a  txqueuelen 1000  (Ethernet)
        RX packets 9847233  bytes 3102031962 (2.8 GiB)
        RX errors 0  dropped 862464  overruns 0  frame 0
        TX packets 556048  bytes 167179960 (159.4 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 369025  bytes 110397861 (105.2 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 369025  bytes 110397861 (105.2 MiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

To find yours current ip, run `ifconfig` and look for the first two numbers `139.147`.

To make the project a bit easier, you are working with a minimal web server, consisting of only a few hundred lines of C code. As a result, the server is limited in its functionality; it does not handle any HTTP requests other than `GET`, understands only a few content types, and supports only the `QUERY_STRING` environment variable for CGI programs. This web server is also not very robust; for example, if a web client closes its connection to the server, it may trip an assertion in the server causing it to exit. We do not expect you to fix these problems (though you can, if you like, you know, for fun).

Helper functions are provided to simplify error checking. A wrapper calls the desired function and immediately terminates if an error occurs. The wrappers are found in the file `io-helper.h`; more about this below. One should always check error codes, even if all you do in response is exit; dropping errors silently is bad C programming and should be avoided at all costs.

2.3 Security

Running a networked server can be dangerous, especially if you are not careful. Thus, security is something you should consider carefully when creating a web server. One thing you should always make sure to do is not leave your server running beyond testing, thus leaving open a potential backdoor into files in your system.

Your system should also make sure to constrain file requests to stay within the sub-tree of the file system hierarchy, rooted at the base working directory that the server starts in. You must take steps to ensure that pathnames that are passed in do not refer to files outside of this sub-tree. One simple (perhaps overly conservative) way to do this is to reject any pathname with `..` in it, thus avoiding any traversals up the file system tree. More sophisticated solutions could use `chroot()` or Linux containers, but perhaps those are beyond the scope of the project.

2.4 Command-line Parameters

Your C program must be invoked exactly as follows:

```
$ ./wserver [-d basedir] [-p port]
```

The command line arguments to your web server are to be interpreted as follows.

- `basedir`: The root directory from where the web server should operate. The server should try to ensure that file accesses do not access files above this directory in the file-system hierarchy. Default: current working directory (e.g., '`.`').
- `port`: The port number that the web server should listen on; the basic web server already handles this argument. Default: 10000.

For example, you could run your program as:

```
$ wserver -d . -p 8003 -t 8 -b 16 -s SFF
```

In this case, the web server will use the current directory and listen to port 8003.

2.5 Source Code Overview

It is recommended you have an understanding how the provided code works. So the following overview is provided to get you started.

- `wserver.c`: Contains 'main()' for the web server and the basic serving loop.
- `request.c`: Performs most of the work for handling requests in the basic web server. Start at `request_handle()` and work through the logic from there.
- `io_helper.h` and `io_helper.c`: Contains wrapper functions for the system calls invoked by the basic web server and client. The convention is to add `_or_die` to an existing call to provide a version that either succeeds or exits. For example, the `open()` system call is used to open a file, but can fail for a number of reasons. The wrapper, `open_or_die()`, either successfully opens a file or exists upon failure.
- `wclient.c`: Contains `main()` and the support routines for the very simple web client. To test your server, you may want to change this code so that it can send simultaneous requests to your server. By launching `wclient` multiple times, you can test how your server handles concurrent requests.
- `spin.c`: A simple CGI program. Basically, it spins for a fixed amount of time, which you may useful in testing various aspects of your server. This will form the basis of your SQL server.
- `Makefile`: You are also provided with a sample Makefile that creates `wserver`, `wclient`, and `spin.cgi`. You can type make to create all of these programs. You can type make clean to remove the object files and the executables. You can type make server to create just the server program, etc. As you create new files, you will ne ed to add them to the `Makefile`.

The best way to learn about the code is to compile it and run it. Run the server we gave you with your preferred web browser. Run this server with the client code we gave you. You can even have the client code we gave you contact any other server that speaks HTTP. Make small changes to the server code (e.g., have it print out more debugging information) to see if you understand how it works.

3 The SQL Server – The Overview

Described above the goal of this project is to create an integrated SQL server, which will be done by creating a Common Gateway Iterface that the webserver will pass commands to.

First, since the sever will run and perform a single command and end, then the table schemas and records need to be stored in a file. **There can be no persistent memory beyond what is stored in the files.**

Second, to make this data storage efficient, the files will be structured into blocks of 256 bytes that can be accessed using the `lseek` file command. Thus, you will be able to move to arbitrary locations in a file: create a new block, retrieve from or write over an existing block.

Third, if an entire file is read into a memory buffer, or a `struct` is used to organize a block, the project will loose a significant number of points.

To make life a little easier, you **cannot** use variable length character strings in your SQL tables. Also, all data stored in the file will be as a string of characters. So if you have a number, say '345', do not store it as the binary representation, convert it to a string of characters with leading zeros, like '0000345'. *This is all to say, do not get fancy.*

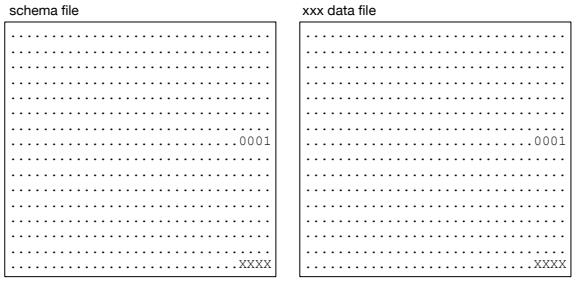
Two hints:

1. `sprintf` and `sscanf` are your pals.
2. The format string you pass does not need to be statically coded and can be dynamically created.

3.1 Design/Implementation Requirements

The focus of the SQL server is on the creation of data storage blocks and using them to build up database table storage. These blocks will all be all be a common 256B and stored in a file sequentially. Blocks can be linked by storing the next location at the end of the current block, with the first block starting in location 0 of the file. This allows for a chain of blocks to be created in the file where the data is stored. So if all the items stored in a block are deleted, then the block can be marked as empty and the chain modified to reflect this. To keep things simple, you can assume that a single table record will not exceed the size of a single block, and if a block is not fully utilized then there can be empty space at the end. Again, for simplicity, all records will be stored in an unordered format, which is to indicate, records are stored in the order they were inserted into the table.

Each created table will be stored in a single file. Tables are tracked using a single file representing the master table storing each table schema in some way that allows for verification of the organization of the data being stored. This information will also be organized into 256B blocks, using a chaining mechanism. Here are two example files with two empty blocks, with the value "XXXX" indicating there is not a next block.



All table records will be stored as a fixed width string. If you wish there can be separator character to facilitate the manipulation of a block. But the following are the data types that you are required to implement in your SQL server.

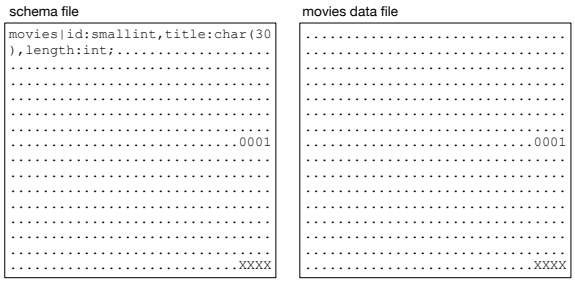
`char(n)`	fixed-length, blank padded
`smallint`	4-byte integer
`integer`	8-byte integer

3.2 Required SQL commands

You are required to provide the following SQL commands: **CREATE**, **INSERT**, **UPDATE**, **SELECT**, and **DELETE**
An example **table create** SQL statement is provided. Also, the order given in the functional call will be the order that data is stored in the tuple.

```
sql statement ==> CREATE TABLE movies (
                     id         smallint,
                     title      char(30),
                     length     int,
                  );
```

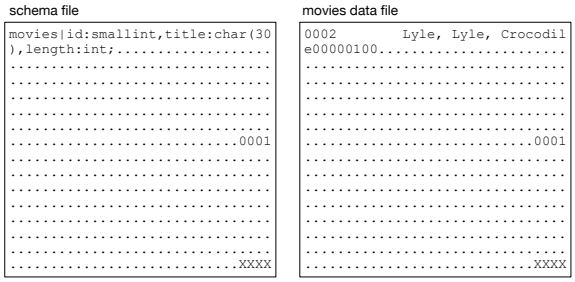
This will create something like the following files:



An example **insert** SQL statement is provided. This function can be kept simple, with the data stored in the order it is positioned in the tuple. When decoding the data, the structure associated with the table create can be used to decode the passed information. Note, you are required to follow the associated example insert, thus you cannot provide the ordered attributes from this function call.

```
sql statement ==> INSERT INTO movies VALUES (2, 'Lyle, Lyle, Crocodile', 100);
```

This will create something like the following files:



An example **update** SQL statement is provided. In this case the variable parameters match the attribute order specified in the table create . The `where` clause will implement: equivalence, not-equal, greater-than inequality, and less-than inequality. You are not required to implement the `AND` and `OR` operators, but doing so will be a boost.

```
sql statement ==> UPDATE movies SET length = 150 WHERE id = 2;
```

This will create something like the following files:

schema file

```
movies|id:smallint,title:char(30
),length:int;...................
................................
................................
................................
................................
................................
.............................0001
................................
................................
................................
................................
................................
................................
................................
.............................XXXX
```

movies data file

```
0002        Lyle, Lyle, Crocodil
e00000250.......................
................................
................................
................................
................................
................................
.............................0001
................................
................................
................................
................................
................................
................................
................................
.............................XXXX
```

An example **select** SQL statement is provided. The requirements for the where clause will follow that for the update command. Aggregate functions are not required to be implemented, nor are the ordering/grouping clauses.

```
sql statement ==> SELECT title, length
                    FROM movies
                   WHERE length < 50;
```

An example several **insert**s and then a **delete** SQL statements is provided. The delete will remove records from a table, by either marking that location as empty or rewriting the entire block with the identified records removed. The while clause should be have similar to the **update** and **select** commands.

```
sql statement ==> INSERT INTO movies VALUES (0, 'The Electric State', 100);
                  INSERT INTO movies VALUES (3, 'Wicked', 150);
                  INSERT INTO movies VALUES (5, 'A Complete Unknown', 230);
                  DELETE FROM movies WHERE length = 100;
```

This will create something like the following files:

movies data file initial

```
0002        Lyle, Lyle, Crocodil
e00000250.......................
................................
................................
................................
................................
................................
.............................0001
................................
................................
................................
................................
................................
................................
................................
.............................XXXX
```

movies data file after inserts

```
0002        Lyle, Lyle, Crocodil
e000002500000            The Ele
ctric State000001000003
               Wicked00000150000
5            A Complete Unknown0
0000230........................
................................
.............................0001
................................
................................
................................
................................
................................
................................
................................
.............................XXXX
```

movies data file after delete (one approach)

```
0002        Lyle, Lyle, Crocodil
e00000250xxxxxxxxxxxxxxxxxxxxxxxx
xxxxxxxxxxxxxxxxxxxxxxxxxxxx0003
               Wicked00000150000
5            A Complete Unknown0
0000230........................
................................
.............................0001
................................
................................
................................
................................
................................
................................
................................
.............................XXXX
```

movies data file after delete (other approach)

```
0002        Lyle, Lyle, Crocodil
e000002500003               Wick
ed000001500005            A Comp
lete Unknown00000230............
................................
.............................0001
................................
................................
................................
................................
................................
................................
.............................XXXX
```

3.3 Error Handling and Valid Processing

It is required that error codes be return to the webserver when something goes incorrectly, such a table is requested that does not exist. These errors will be passed back to the client as a normal CGI value. If a select correctly retrieves data, then this data will be returned back.

Solution grading

The assignment will have the following percentage breakdown and will be graded on a 4-point Lafayette College GPA scale.

Aspect	Percent	Item
Implementation	10%	Code organization, comments, and clarity of design.
	10%	Ease of use during grading.
	10%	Organization and understandability of system-dump.
Verification	10%	Implementation of testing mechanism and understandability of result.
	10%	Completeness of tests.
	10%	Correctness of test.
Final Report	10%	Description of implemented software.
	10%	Algorithms used and their implementation.
	10%	Verification plan and demonstration of that plan.
	10%	Usage documentation, describing how to run the system.

A Basic File Handling Example

Below is an example of a program that uses the low level file handling mechanisms to read and write to a file. The most interesting part is the use of `lseek` to move about the file.

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/uio.h>
#include <unistd.h>

int main(int argc, char *argv[]) {

  // create initial buffer
  char buf[32] = "                            0001";

  // open file for write only, and create the file
  // -- then write the buffer to the file and close
  int fd = open("table_example.txt", O_WRONLY | O_CREAT);
  write(fd, buf, 32);
  close(fd);

  // open file for write only, then move to the fourth byte
  // write the value "1234" and close file
  fd = open("table_example.txt", O_WRONLY);
  lseek(fd, 4, SEEK_SET);
  write(fd, "1234", 4);
  close(fd);

  // open file for read only and move to the fourth byte
  // read two bytes from the file into the buffer and
  // null-terminate the read information
  // then close the file
  fd = open("table_example.txt", O_RDONLY);
  lseek(fd, 4, SEEK_SET);
  read(fd, buf, 2);
```

```
    buf[2] = '\0';
    close(fd);

    // print the string in the buffer
    puts(buf);

    return 0;
  }
```

If the program is run, we will see the following. Performing a more on the file to see its contents can also be seen here.

```
% gcc -o example example.c
% ./example
12

% ls -la
total 88
drwxr-xr-x@  5 pfaffmaj  staff    160 Apr  8 07:03 .
drwxr-xr-x@ 15 pfaffmaj  staff    480 Apr  8 06:26 ..
-rwxr-xr-x@  1 pfaffmaj  staff  33752 Apr  8 07:02 example
-rw-r--r--@  1 pfaffmaj  staff    573 Apr  8 07:03 example.c
-rw------x@  1 pfaffmaj  staff     32 Apr  8 07:02 table_example.txt

% more table_example.txt
    1234                    0001
```

To find out more about these commands, use the following man pages.

1. `man 2 open`
2. `man 2 read`
3. `man 2 write`
4. `man 2 lseek`